mtblc

Generated by Doxygen 1.16.1

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 4

Class Documentation

4.1 __SMtLConfigParam Struct Reference

Public Attributes

- unsigned int **nDeviceTimeOut** =65535

The documentation for this struct was generated from the following file:

- mtblc.h

4.2 __SMtLEyeCBInfo Struct Reference

Public Attributes

- unsigned int **nSize**
- char **byServerIP** [128]

The documentation for this struct was generated from the following file:

- mtblc.h

4.3 __SMtPoint3f Struct Reference

Public Attributes

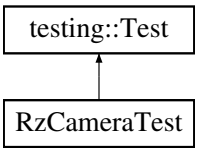
- float **x**
- float **y**
- float **z**

The documentation for this struct was generated from the following file:

- mtblc.h

4.4 RzCameraTest Class Reference

Inheritance diagram for RzCameraTest:



Public Attributes

- std::vector< std::shared_ptr< Camera > > **cams**
- std::shared_ptr< RzFpga > **rzfpga** =nullptr

Protected Member Functions

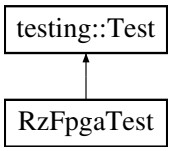
- void **SetUp** () override
- void **TearDown** () override

The documentation for this class was generated from the following file:

- test_rz.cc

4.5 RzFpgaTest Class Reference

Inheritance diagram for RzFpgaTest:



Public Attributes

- std::shared_ptr< RzFpga > **rzfpga**

Protected Member Functions

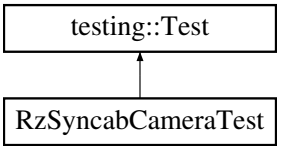
- void **SetUp** () override
- void **TearDown** () override
- cv::Mat **GetCamAB** ()

The documentation for this class was generated from the following file:

- test_rz.cc

4.6 RzSyncabCameraTest Class Reference

Inheritance diagram for RzSyncabCameraTest:



Public Attributes

- std::shared_ptr< RzFpga > **rzfpga** =nullptr
- std::vector< std::shared_ptr< Camera > > **cams**

Protected Member Functions

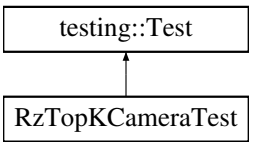
- void **SetUp** () override
- void **TearDown** () override

The documentation for this class was generated from the following file:

- test_rz.cc

4.7 RzTopKCameraTest Class Reference

Inheritance diagram for RzTopKCameraTest:



Public Attributes

- std::vector< std::shared_ptr< Camera > > **cams**
- std::shared_ptr< RzFpga > **rzfpga** =nullptr

Protected Member Functions

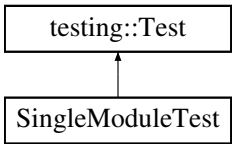
- void **SetUp** () override
- void **TearDown** () override

The documentation for this class was generated from the following file:

- test_rz.cc

4.8 SingleModuleTest Class Reference

Inheritance diagram for SingleModuleTest:



Protected Member Functions

- void **SetUp** () override
- void **TearDown** () override

Protected Attributes

- MTLHANDLE **hDevice** =-1

The documentation for this class was generated from the following file:

- test_mtblc.cc

4.9 SMtPointAttachment Struct Reference

Attachment data for individual points in a point cloud.

`#include <mtblc.h>`

Public Attributes

- int **timestamp**

  Timestamp of when the point was captured.
- int **frameid**

  Frame ID of the point cloud.
- float **r**
- float **g**
- float **b**

  RGB color values for the point cloud.

4.9.1 Detailed Description

Attachment data for individual points in a point cloud.

Contains additional metadata for each point including frame information and color data.

The documentation for this struct was generated from the following file:

- mtblc.h

4.10 SMtPointcloud Struct Reference

Store point cloud data.

`#include <mtblc.h>`

Public Attributes

- SMtPoint3f * **coords** =nullptr
  Array of 3D point coordinates.
- size_t **num_points** =0
  Number of points in the point cloud.

4.10.1 Detailed Description

Store point cloud data.

The documentation for this struct was generated from the following file:

- mtblc.h

4.11 SMtPointcloudAttachment Struct Reference

Collection of attachment data for all points in a point cloud.

`#include <mtblc.h>`

Public Attributes

- SMtPointAttachment * **data** =nullptr
  Array of attachment data for each point.
- size_t **num_points** =0
  Number of attachment entries.
- size_t **number_of_bytes_per_attachment** =sizeof(SMtPointAttachment)
  Size of each attachment entry for future extensibility and version verification.

4.11.1 Detailed Description

Collection of attachment data for all points in a point cloud.

Contains an array of attachment data corresponding to points in the point cloud, along with size information for verification and extensibility.

The documentation for this struct was generated from the following file:

- mtblc.h

4.12 SMtScanConfig Struct Reference

Configuration parameters for laser scanning operations.

`#include <mtblc.h>`

Public Attributes

- double **speed_scan** =10

  Scanning speed in units per second.
- double **range_deg** =50

  Scanning range in degrees.
- uint16_t **exposure_time_us** =100

  Camera exposure time in microseconds.
- uint8_t **laserpower** =200

  Laser power level [1,250].
- bool **color_camera** =true

  Whether to capture color information with the point cloud.

4.12.1 Detailed Description

Configuration parameters for laser scanning operations.

This structure contains all the parameters needed to configure a laser scan, including motor speed, scanning range, exposure settings, and laser power.

The documentation for this struct was generated from the following file:

- mtblc.h

Chapter 5

File Documentation

5.1 mtblc.h File Reference

Motic SDK common definitions and APIs.

```
#include "inc/MTL_Export.h"
```

Classes

- struct __SMtPoint3f
- struct __SMtLConfigParam
- struct __SMtLEyeCBInfo
- struct SMtScanConfig

  Configuration parameters for laser scanning operations.
- struct SMtPointcloud

  Store point cloud data.
- struct SMtPointAttachment

  Attachment data for individual points in a point cloud.
- struct SMtPointcloudAttachment

  Collection of attachment data for all points in a point cloud.

Typedefs

- typedef int **MTLHANDLE**
- typedef struct __SMtPoint3f **SMtPoint3f**
- typedef struct __SMtLConfigParam **SMtLConfigParam**
- typedef struct __SMtLEyeCBInfo **SMtLEyeCBInfo**

Functions

- MTLAPI int MtL_Init (const SMtLConfigParam *pConfigParam)

  Initialize the Motic laser scanning library.
- MTLAPI int MtL_SearchDevices ()

  Search for devices.
- MTLAPI MTLHANDLE MtL_OpenDevice (int device_id)

  Open device.
- MTLAPI int MtL_CloseDevice (MTLHANDLE hDevice)

  Close device.
- MTLAPI int MtL_GetDeviceInfo (MTLHANDLE hDevice, SMtLEyeCBInfo *pInfo)

  Get device information.
- MTLAPI int MtL_GetFilteredPointcloud (SMtPointcloud *pPointcloud_in, SMtPointcloud *pPointcloud_out)

  Get a filtered version of a point cloud.
- MTLAPI int MtL_DestroyPointcloud (SMtPointcloud *pPointcloud)

  Destroy and free memory allocated for a point cloud structure.
- MTLAPI int MtL_DestroyPointcloudAttachment (SMtPointcloudAttachment *pPointcloudAttachment)

  Destroy and free memory allocated for point cloud attachment data.
- MTLAPI int MtL_Lidflip (MTLHANDLE hDevice)

  Flip or toggle the laser scanning device lid.
- MTLAPI int MtL_Scan (MTLHANDLE hDevice, SMtScanConfig *pScanConfig, SMtPointcloud *pPointcloud, SMtPointcloudAttachment *pPointcloudAttachment=nullptr)

  Perform a synchronous laser scan operation.
- MTLAPI int MtL_ScanAsyncStart (MTLHANDLE hDevice, SMtScanConfig *pScanConfig, bool *pCtrl↩ Flag, void(*pScanCB)(SMtPointcloud *pPointcloud, SMtPointcloudAttachment *pAttachment, void *ctx), void *userContext)

  Start an asynchronous laser scan operation.
- MTLAPI int MtL_ScanAsyncStop (MTLHANDLE hDevice, bool *pCtrlFlag)

  Stop an ongoing asynchronous laser scan operation.
- MTLAPI int MtL_MapTexture (MTLHANDLE hDevice, const char *path_calibAB, const char *path_calibAC, SMtPointcloud *pPointcloud, SMtPointcloudAttachment *pPointcloudAttachment)

  Map texture from the color camera (C) onto a point cloud.
- MTLAPI int MtL_GetMotorPosition (MTLHANDLE hDevice, double *pPosition)

  Obtain the current position of the motor in degrees. The position is measured in degrees, where 0 degrees corresponds to the zero position of the motor, and positive values indicate rotation in the forward direction. This function can be used to monitor the motor's position during scanning or to verify that the motor has reached a specific position after a move command.
- MTLAPI int MtL_EnableLaser (MTLHANDLE hDevice, bool bEnable)

  Enable or disable the laser. Enabling the laser will turn it on if the motor is moving, and keep it on until the motor stops. Disabling the laser will turn it off immediately. This function can be used to control whether the laser is on during motor scanning, or to manually turn on the laser for other purposes (e.g. for visual inspection) without starting a scan.
- MTLAPI char * **MtL_GetLastErrorMessage** ()
- MTLAPI void **MtL_GetErrorInfo** (int nErrorCode, char szError[256])
- MTLAPI void **MtL_Destroy** ()

  Destroy and cleanup the SDK.

5.1.1 Detailed Description

Motic SDK common definitions and APIs.

This file includes data structures and functions of Motic SDK for laser scanning

```
MtL_Init();
// ... your code here ...
MtL_Destroy();
```

Date

5.1.2 Function Documentation

5.1.2.1 MtL_CloseDevice()

```
MTLAPI int MtL_CloseDevice (
            MTLHANDLE hDevice)
```

Close device.

Parameters

in	*hDevice*	Device handle

Returns

Returns 0 on successful close, otherwise error code

5.1.2.2 MtL_DestroyPointcloud()

```
MTLAPI int MtL_DestroyPointcloud (
            SMtPointcloud * pPointcloud)
```

Destroy and free memory allocated for a point cloud structure.

This function releases all memory associated with a point cloud, including the coordinate array. After calling this function, the point cloud structure should not be used until it is re-initialized.

Parameters

in,out	*pPointcloud*	Pointer to the point cloud structure to destroy

Returns

An integer error code. 0 indicates success, while non-zero indicates failure

Return values

0	Indicates success
Non-zero	Indicates failure, you can use MtL_GetErrorInfo to get more information

5.1.2.3 MtL_DestroyPointcloudAttachment()

```
MTLAPI int MtL_DestroyPointcloudAttachment (
            SMtPointcloudAttachment * pPointcloudAttachment)
```

Destroy and free memory allocated for point cloud attachment data.

This function releases all memory associated with point cloud attachment data, including the attachment array. After calling this function, the attachment structure should not be used until it is re-initialized.

Parameters

in,out	*pPointcloudAttachement*	Pointer to the point cloud attachment structure to destroy

Returns

An integer error code. 0 indicates success, while non-zero indicates failure

Return values

0	Indicates success
Non-zero	Indicates failure, you can use MtL_GetErrorInfo to get more information

5.1.2.4 MtL_EnableLaser()

```
MTLAPI int MtL_EnableLaser (
            MTLHANDLE hDevice,
            bool bEnable)
```

Enable or disable the laser. Enabling the laser will turn it on if the motor is moving, and keep it on until the motor stops. Disabling the laser will turn it off immediately. This function can be used to control whether the laser is on during motor scanning, or to manually turn on the laser for other purposes (e.g. for visual inspection) without starting a scan.

Parameters

in	*hDevice*	Device handle
in	*bEnable*	Enable or disable the laser

Returns

An integer error code. 0 indicates success, while non-zero indicates failure. In case of failure, you can use the MtL_GetErrorInfo function to get more information about the error.

Return values

0	Indicates success
Non-zero	Indicates failure, you can use MtL_GetErrorInfo to get more information

5.1.2.5 MtL_GetDeviceInfo()

```
MTLAPI int MtL_GetDeviceInfo (
             MTLHANDLE hDevice,
             SMtLEyeCBInfo * pInfo)
```

Get device information.

Parameters

in	*hDevice*	Device handle
in	*pInfo*	Device information

Returns

Returns 0 on success, otherwise error code

5.1.2.6 MtL_GetFilteredPointcloud()

```
MTLAPI int MtL_GetFilteredPointcloud (
             SMtPointcloud * pPointcloud_in,
             SMtPointcloud * pPointcloud_out)
```

Get a filtered version of a point cloud.

This function applies a filtering algorithm to the input point cloud and produces a new point cloud containing only the filtered points.

Parameters

in	*pPointcloud_in*	Pointer to the input point cloud
out	*pPointcloud_out*	Pointer to the output point cloud that will receive the filtered points

Returns

An integer error code. 0 indicates success, while non-zero indicates failure

Return values

0	Indicates success
Non-zero	Indicates failure, you can use MtL_GetErrorInfo to get more information

5.1.2.7 MtL_GetMotorPosition()

```
MTLAPI int MtL_GetMotorPosition (
            MTLHANDLE hDevice,
            double * pPosition)
```

Obtain the current position of the motor in degrees. The position is measured in degrees, where 0 degrees corresponds to the zero position of the motor, and positive values indicate rotation in the forward direction. This function can be used to monitor the motor's position during scanning or to verify that the motor has reached a specific position after a move command.

Parameters

in	*hDevice*	Device handle
out	*pPosition*	Pointer to a double where the current motor position in degrees will be stored

Returns

An integer error code. 0 indicates success, while non-zero indicates failure. In case of failure, you can use the MtL_GetErrorInfo function to get more information about the error.

Return values

0	Indicates success
Non-zero	Indicates failure, you can use MtL_GetErrorInfo to get more information

5.1.2.8 MtL_Init()

```
MTLAPI int MtL_Init (
            const SMtLConfigParam * pConfigParam)
```

Initialize the Motic laser scanning library.

Parameters

in	*pConfigParam*	Configuration parameters for initializing the library. If nullptr, default parameters will be used.

Returns

An integer error code. 0 indicates success, while non-zero indicates failure

5.1.2.9 MtL_Lidflip()

```
MTLAPI int MtL_Lidflip (
            MTLHANDLE hDevice)
```

Flip or toggle the laser scanning device lid.

This function controls the mechanical lid of the laser scanning device, typically used to open or close the scanning chamber for sample placement or removal.

Parameters

in	*hDevice*	Device handle for the laser scanning device

Returns

An integer error code. 0 indicates success, while non-zero indicates failure

Return values

0	Indicates success
Non-zero	Indicates failure, you can use MtL_GetErrorInfo to get more information

5.1.2.10 MtL_MapTexture()

```
MTLAPI int MtL_MapTexture (
             MTLHANDLE hDevice,
             const char * path_calibAB,
             const char * path_calibAC,
             SMtPointcloud * pPointcloud,
             SMtPointcloudAttachment * pPointcloudAttachment)
```

Map texture from the color camera (C) onto a point cloud.

Parameters

in	*calib_AB*	path to calibration file for monochrome cameras
in	*calib_AC*	path to calibration file for the color camera (with respect to the first monochrome camera)
in	*pPointcloud*	point cloud obtained from a previous scan
in,out	*pPointcloudAttachments*	additional point cloud data or attachments

Return values

Non-zero	Indicates failure, you can use MtL_GetErrorInfo to get more information

5.1.2.11 MtL_OpenDevice()

```
MTLAPI MTLHANDLE MtL_OpenDevice (
            int device_id)
```

Open device.

Parameters

in	*hDevice*	Device handle
out	*pOpenDeviceParam*	Device open information
out	*pErrorCode*	Returns error code, does not return when NULL

Returns

Returns 0 on success, otherwise error code

5.1.2.12 MtL_Scan()

```
MTLAPI int MtL_Scan (
            MTLHANDLE hDevice,
            SMtScanConfig * pScanConfig,
            SMtPointcloud * pPointcloud,
            SMtPointcloudAttachment * pPointcloudAttachment = nullptr)
```

Perform a synchronous laser scan operation.

This function executes a complete laser scan using the specified configuration parameters and returns the resulting point cloud data. The function blocks until the scan is complete. Optionally, attachment data with additional metadata can also be retrieved.

Parameters

in	*hDevice*	Device handle for the laser scanning device
in	*pScanConfig*	Pointer to scan configuration parameters
out	*pPointcloud*	Pointer to structure that will receive the point cloud data
out	*pPointcloudAttachment*	Optional pointer to structure that will receive attachment data (can be nullptr)

Returns

An integer error code. 0 indicates success, while non-zero indicates failure

Return values

0	Indicates success
Non-zero	Indicates failure, you can use MtL_GetErrorInfo to get more information

5.1.2.13 MtL_ScanAsyncStart()

```
MTLAPI int MtL_ScanAsyncStart (
            MTLHANDLE hDevice,
            SMtScanConfig * pScanConfig,
            bool * pCtrlFlag,
            void(* pScanCB )(SMtPointcloud *pPointcloud, SMtPointcloudAttachment *pAttachment,
void *ctx),
            void * userContext)
```

Start an asynchronous laser scan operation.

This function initiates a non-blocking laser scan that runs in the background. Point cloud data is delivered through a callback function as it becomes available. The scan continues until stopped via MtL_ScanAsyncStop or the control flag is set to false.

Parameters

in	*hDevice*	Device handle for the laser scanning device
in	*pScanConfig*	Pointer to scan configuration parameters
in,out	*pCtrlFlag*	Pointer to boolean flag for controlling scan execution (set to false to stop)
in	*pScanCB*	Callback function that receives point cloud data as it's generated
in	*userContext*	User-defined context pointer passed to the callback function

Returns

An integer error code. 0 indicates success, while non-zero indicates failure

Return values

0	Indicates success
Non-zero	Indicates failure, you can use MtL_GetErrorInfo to get more information

5.1.2.14 MtL_ScanAsyncStop()

```
MTLAPI int MtL_ScanAsyncStop (
            MTLHANDLE hDevice,
            bool * pCtrlFlag)
```

Stop an ongoing asynchronous laser scan operation.

This function terminates an active asynchronous scan that was started with MtL_ScanAsyncStart. It sets the control flag to false and waits for the scan thread to complete gracefully.

Parameters

in	*hDevice*	Device handle for the laser scanning device
in,out	*pCtrlFlag*	Pointer to the same control flag used in MtL_ScanAsyncStart

Returns

An integer error code. 0 indicates success, while non-zero indicates failure

Return values

0	Indicates success
Non-zero	Indicates failure, you can use MtL_GetErrorInfo to get more information

5.1.2.15 MtL_SearchDevices()

```
MTLAPI int MtL_SearchDevices ()
```

Search for devices.

Returns

number of devices found

5.2 mtblc.h

Go to the documentation of this file.

```
00001
00014
00015 #define MOTICSDK_EXPORTS
00016 #pragma warning( disable : 4201 ) //disable warning on nameless struct/union because they are used
      widely in vizum headers, which we have to simulate here
00017 #include "inc/MTL_Export.h"
00018 typedef int MTLHANDLE;
00019 typedef struct __SMtPoint3f { float x; float y; float z; } SMtPoint3f;
00020 typedef struct __SMtLConfigParam{ unsigned int        nDeviceTimeOut=65535;  }    SMtLConfigParam;
00021 typedef struct __SMtLEyeCBInfo { unsigned int       nSize; char byServerIP[128];} SMtLEyeCBInfo;
00022 #pragma warning( default : 4201 )
00023
00030 struct SMtScanConfig
00031 {
00032     double speed_scan=10;
00033     double range_deg=50;
00034     uint16_t exposure_time_us=100;
00035     uint8_t laserpower=200;
00036     bool color_camera=true;
00037 };
00038
00042 struct SMtPointcloud
00043 {
00044     SMtPoint3f* coords=nullptr;
00045     size_t num_points=0;
00046 };
00047
00053 struct SMtPointAttachment
00054 {
00055     int timestamp;
00056     int frameid;
00057     float r,g,b;
00058 };
00059
00066 struct SMtPointcloudAttachment
00067 {
00068     SMtPointAttachment* data=nullptr;
00069     size_t num_points=0;
00070     size_t number_of_bytes_per_attachment=sizeof(SMtPointAttachment);
00071 };
00077 MTLAPI int MtL_Init(const SMtLConfigParam* pConfigParam);
00078
00083 MTLAPI int MtL_SearchDevices();
00084
00092 MTLAPI MTLHANDLE MtL_OpenDevice(int device_id);
00093
```

```
00099 MTLAPI int MtL_CloseDevice(MTLHANDLE hDevice);
00100
00107 MTLAPI int MtL_GetDeviceInfo(MTLHANDLE hDevice, SMtLEyeCBInfo* pInfo);
00120 MTLAPI int MtL_GetFilteredPointcloud(SMtPointcloud* pPointcloud_in,SMtPointcloud* pPointcloud_out);
00133 MTLAPI int MtL_DestroyPointcloud(SMtPointcloud* pPointcloud);
00134
00147 MTLAPI int MtL_DestroyPointcloudAttachment(SMtPointcloudAttachment* pPointcloudAttachment);
00148
00161 MTLAPI int MtL_Lidflip(MTLHANDLE hDevice);
00162
00179 MTLAPI int MtL_Scan(MTLHANDLE hDevice,SMtScanConfig* pScanConfig,SMtPointcloud* pPointcloud,
      SMtPointcloudAttachment* pPointcloudAttachment=nullptr);
00180
00197 MTLAPI int MtL_ScanAsyncStart(MTLHANDLE hDevice,SMtScanConfig* pScanConfig,bool*
      pCtrlFlag,void(*pScanCB)(SMtPointcloud* pPointcloud,SMtPointcloudAttachment* pAttachment, void* ctx),
      void* userContext);
00198
00212 MTLAPI int MtL_ScanAsyncStop(MTLHANDLE hDevice,bool* pCtrlFlag);
00213
00223 MTLAPI int MtL_MapTexture(MTLHANDLE hDevice,const char* path_calibAB,const char*
      path_calibAC,SMtPointcloud* pPointcloud,SMtPointcloudAttachment* pPointcloudAttachment);
00224
00233 MTLAPI int MtL_GetMotorPosition(MTLHANDLE hDevice,double* pPosition);
00234
00243 MTLAPI int MtL_EnableLaser(MTLHANDLE hDevice, bool bEnable);
00244 MTLAPI char* MtL_GetLastErrorMessage();
00245 /*
00246 * @brief Get error information corresponding to an error code. This function can be used to convert an
      error code returned by other functions into a human-readable error message for debugging or logging
      purposes.
00247 * @param [in] nErrorCode error code returned by a function in case of failure
00248 * @param [out] szError  textual description of the error corresponding to the error code, stored in a
      character array of size 256. The caller should ensure that the array is allocated and has enough space
      to hold the error message.
00249 */
00250 MTLAPI void MtL_GetErrorInfo(int nErrorCode, char szError[256]);
00254 MTLAPI void MtL_Destroy();
```

Index